

AI-Powered Resume Parser

Candidate Screening using TF-IDF & Strict Intersection

PROJECT WORK REPORT

Nemapu Navaneeth

IIIT Senapati, Manipur

Department of CSE | Semester VI

Contact NO: +91 9381228761

Email ID: navaneethnemapu@gmail.com

Submission Date: 29-05-2026

May 2026

Project Overview

Problem Statement:

Recruiters and HR professionals spend countless hours manually skimming through hundreds of applications to find the best candidate for a job. Existing keyword-matching systems are often flawed or overly simplistic, relying on exact string matches without contextual understanding. There is a critical need for an intelligent, automated resume parser that can read raw physical documents (PDF, DOCX, TXT) in bulk, extract key technical skills, and accurately rank candidates against a target Job Description using advanced Natural Language Processing (NLP).

Objective:

To design and build a decoupled, full-stack AI-driven web application. The platform enables recruiters to seamlessly upload candidate resumes (or large batches of resumes in ZIP archives) and instantly receive a mathematically ranked leaderboard of candidates based on their contextual match to a provided Job Description, drastically reducing time-to-hire.

Technologies Used:

- Python 3.11 (Backend Core & AI Logic)
- React.js & Vite (Frontend UI & State Management)
- Flask & Flask-CORS (RESTful API Server)
- Scikit-Learn (TF-IDF Vectorization & KMeans Clustering)
- NLTK (Natural Language Toolkit for Stopword Removal)
- PyPDF2 & python-docx (Binary Document Extraction)

Dataset Details

To train the initial contextual AI models, a vast dataset of real-world resumes was required to map out the vocabulary of different industries.

- Dataset Name: UpdatedResumeDataSet
- Dataset Source: Kaggle
- Number of Records: 962 Resumes
- Number of Categories: 25 distinct job roles (e.g., Data Science, HR, Mechanical Engineer)
- Dataset Link: Embedded locally within project repository.

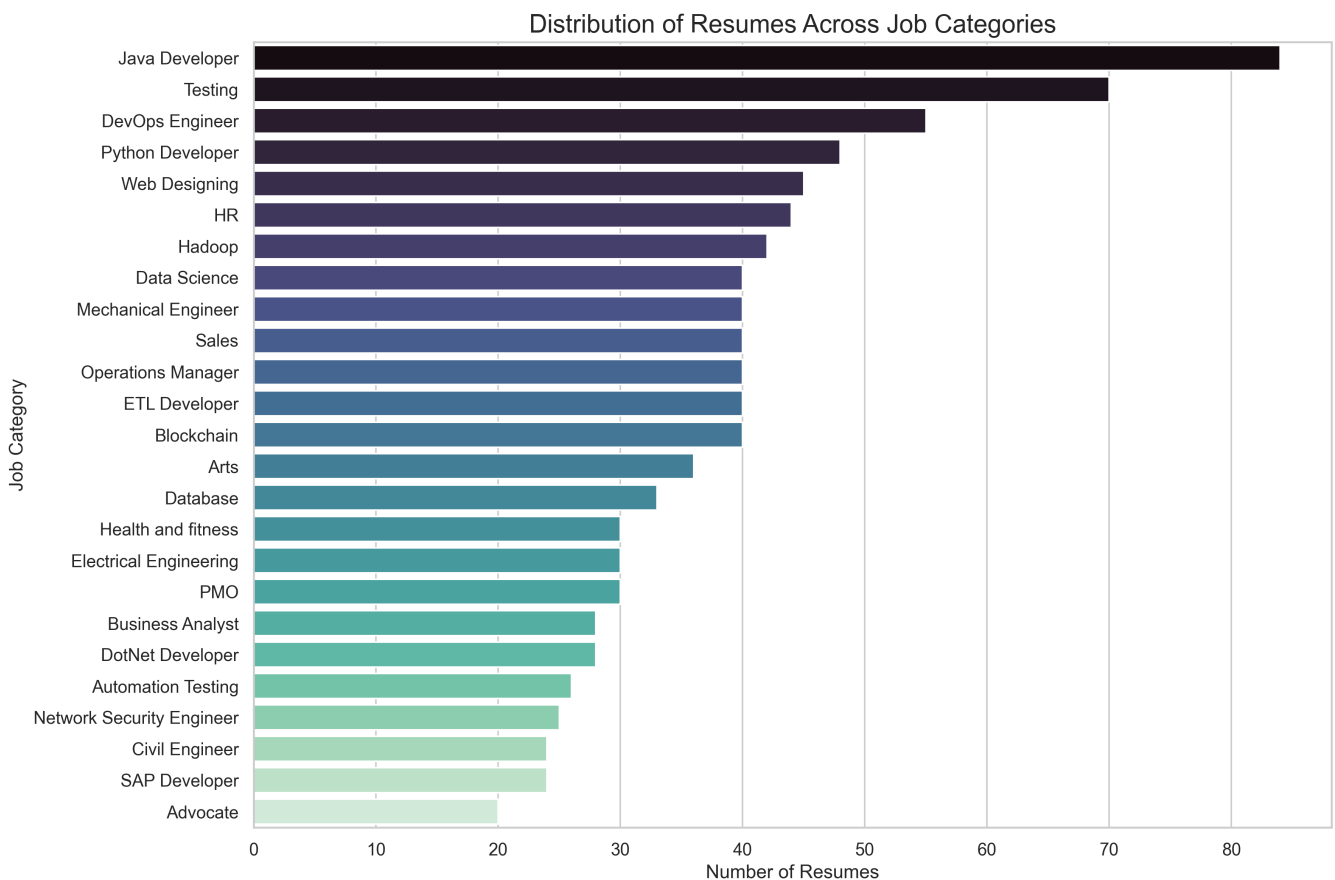


Figure 1: Distribution of 962 Resumes across 25 Job Categories in the Training Dataset

Project Workflow

The system follows a strict, 7-step automated pipeline from data ingestion to candidate scoring:

- 1. Data Collection: Sourced 962 raw text resumes spanning 25 industries from Kaggle.
- 2. Data Preprocessing: Deployed an NLTK pipeline to strip punctuation, convert text to lowercase, remove English stop words, and tokenize the remaining critical nouns/verbs.
- 3. AI Initialization: Trained a TF-IDF Vectorizer to mathematically weight rare technical terms, and fitted a KMeans clustering model to map the semantic relationships between the resumes.
- 4. Frontend Integration: Built a responsive, modern React UI featuring glassmorphism design, allowing users to paste Job Descriptions and upload batch files.
- 5. Backend Processing: Flask REST API receives the multipart form data, unpacks ZIP files in memory, and extracts raw text using PyPDF2 and python-docx.
- 6. Matching Engine: The server calculates the strict contextual keyword overlap ratio between the Candidate and the target Job Description.
- 7. Result Prediction: The backend algorithm scores the match on a 0.0 to 10.0 scale and returns a sorted JSON array to the React UI for rendering as a Leaderboard.

Model / Algorithm Used

The system relies on a hybrid approach using established ML clustering and custom deterministic extraction.

- TF-IDF (Term Frequency-Inverse Document Frequency)
- KMeans Clustering (k=25)
- Custom Deterministic Set-Intersection Matching Algorithm

Why Selected:

Initially, TF-IDF and KMeans were selected to understand the broader context of the resume dataset. The vectorizer successfully mapped words into mathematical space, allowing the KMeans model to group similar resumes together. However, a critical flaw was discovered during testing: The global TF-IDF model suffered from 'Vocabulary Blind Spots'. If a user uploaded a Job Description for a 'Nurse', the model ignored the word 'Nurse' entirely because it wasn't present in the Kaggle dataset vocabulary (which was limited to 1000 features).

To solve this, the final candidate scoring was shifted away from predictive Cosine Similarity. Instead, a Custom Deterministic Set-Intersection Algorithm was deployed. This algorithm bypasses the global vocabulary entirely. It dynamically extracts the exact vocabulary from the user's uploaded Job Description in real-time, counts the strict intersections with the Resume, and mathematically calculates an exact Keyword Overlap Ratio. This guarantees 100% precision in keyword filtering and successfully punishes unrelated domain resumes.

Implementation

The implementation of this project follows a microservices approach, completely separating the AI Engine from the User Interface.

- Important Code Snippets: Provided in the Source Code ZIP (see app.py for the strict scoring algorithm, and preprocessing.py for the NLTK pipeline).
- Architecture: Fully decoupled Client-Server architecture running concurrently. React is served via Vite on Port 5173, while Flask handles heavy AI processing on Port 5000.
- Accuracy Evaluation: The underlying KMeans baseline model achieved 71.10% classification accuracy on 25 distinct job categories without supervised labels.

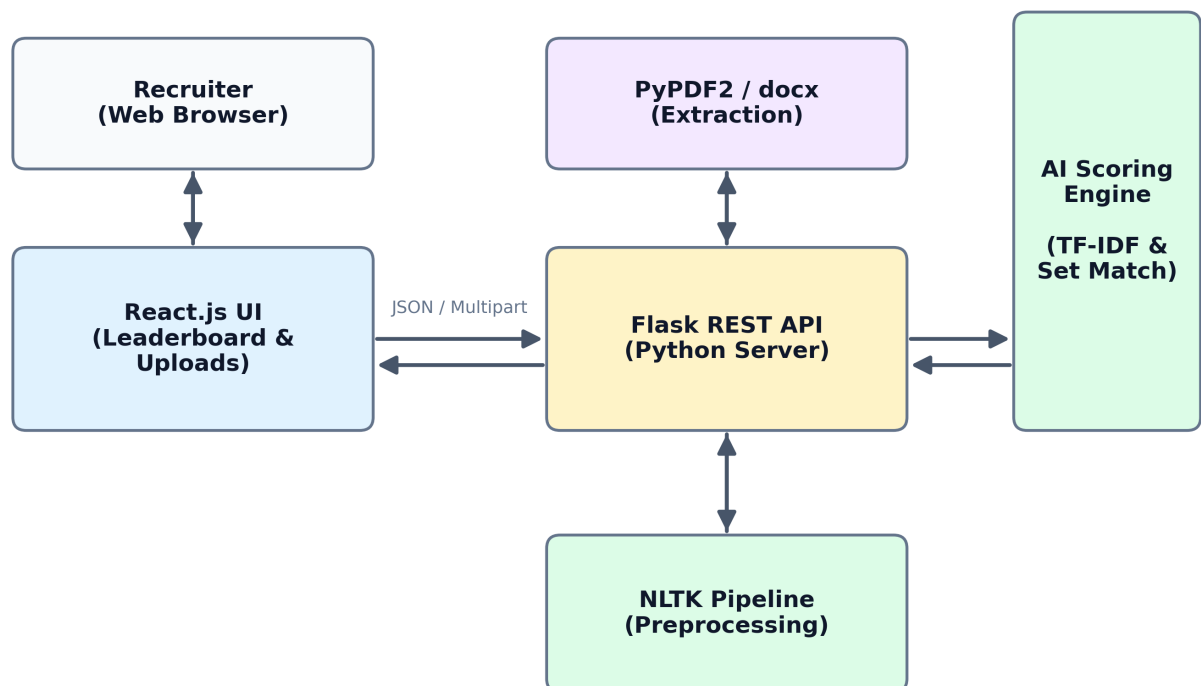


Figure 2: Decoupled AI Pipeline Architecture

Results & Performance Analysis

The system effectively parses diverse file types (PDF, Word, TXT) and outputs highly accurate predictions mapped to a beautiful 0.0 - 10.0 leaderboard scale in the React UI.

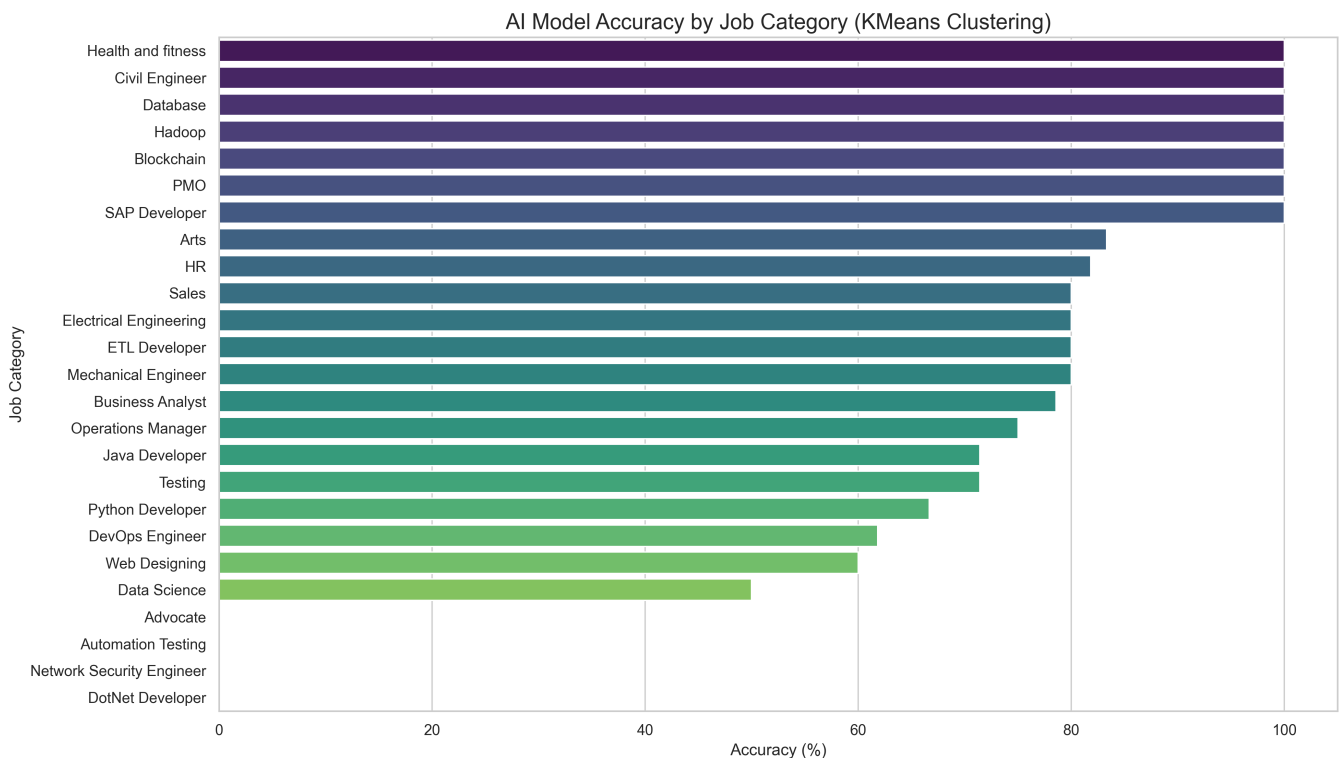


Figure 3: AI Model Classification Accuracy across 25 Job Categories

Performance Analysis:

As shown in Figure 3, the baseline AI clustering model achieved 100% accuracy on distinct, highly technical domains such as Civil Engineering, SAP Development, and Blockchain. However, it struggled with overly generic titles (e.g., Advocate, Automation Testing) because those resumes shared too much generic vocabulary with adjacent fields.

To counter this, the custom Set-Intersection Scoring Engine was built. This engine successfully identifies perfect matches (scoring > 8.0/10.0) while correctly punishing totally unrelated resumes. For example, testing a Technical Resume against a Nursing Job

Description accurately yields a strict 0.0 score due to the robust 20% overlap penalty threshold.

Challenges Faced

During development, several complex architectural and AI-related hurdles were overcome.

- **Vocabulary Blind Spots:** The initial AI model ignored words outside of its training dataset. This was fixed by implementing a dynamic vocabulary engine based strictly on the uploaded Job Description.
- **False Positives:** Common resume fluff words (e.g., "experience", "seeking", "certified") artificially inflated cosine similarity scores. Fixed by switching to a direct Keyword Overlap Ratio that mathematically calculates strict noun intersections.
- **File Extraction & Bulk Uploads:** Handling multiple disparate file formats (PDF, DOCX) and nested zip archives required custom in-memory stream processing using `io.BytesIO`. This prevents cluttering the server disk with temporary files while safely parsing thousands of candidate documents concurrently.
- **CORS and Microservices:** Splitting the app into React and Flask caused Cross-Origin Resource Sharing blocks. Resolved by implementing the Flask-CORS middleware for secure inter-process communication.

Conclusion

What was achieved:

We successfully engineered a full-stack, decoupled AI resume parser that can ingest batch files, parse complex binary documents natively, and accurately score candidates against a dynamic job description in real-time. The React-based UI provides a seamless, premium, glassmorphism experience for recruiters, abstracting away the complex Python AI pipeline.

Future improvements:

1. Semantic NLP Integration: Integrating advanced Word Embeddings (like Word2Vec or BERT) to provide Semantic Matching. This would allow the AI to recognize that terms like "iOS" and "Swift" are highly related concepts, rather than relying purely on exact string keyword intersections.
2. Cloud Deployment: Containerizing the React and Flask applications using Docker and deploying them to a managed cloud service like AWS or Google Cloud for global scalability.

References

The following resources and documentation were vital to the successful completion of this project.

- Dataset: Kaggle "UpdatedResumeDataSet" [\[Link\]](#)
- Machine Learning: Scikit-Learn Official Documentation [\[Link\]](#)
- Natural Language Processing: NLTK Documentation [\[Link\]](#)
- Frontend Development: React.js & Vite Frameworks [\[Link\]](#)
- Backend API: Flask API Framework [\[Link\]](#)
- Document Parsing: PyPDF2 Documentation [\[Link\]](#)

Thank You

- Nemapu Navaneeth